

Section	Description	Points	Grade Breakdown and Requirements			Weightage
		60	What 80-100% looks like	What 60-80% looks like	What <60% looks like	100.00%
Data Overview and Exploratory Data Analysis	- Observations on the shape of data, data types of various attributes, missing values, duplicate values and statistical summary. - Univariate Analysis (boxplots, distribution plots for important variables) - Bivariate Analysis - Insights based on EDA	6	1) Observations on the shape of data, data types of various attributes, missing values, and statistical summary [1] 2) Univariate Analysis (Boxplots and distribution plots for numerical data and barplots for categorical data) [2] 3) Bivariate Analysis (Scatterplots and correlation plot) [2] 4) Insights based on EDA [1]	1) Observations on the shape of data, data types of various attributes, missing values, statistical summary. 2) Univariate Analysis (Boxplots and distribution plots for numerical data and barplots for categorical data)	1) Observations on the shape of data, missing values, statistical summary.	10.00%
Data Preprocessing	- Feature engineering (provide rationale if not needed) - Outlier detection and treatment (provide rationale if treatment is not needed) - Prepare the data for analysis (Train and Test sets) - Define the preprocessing pipeline for encoding categorical features	5	1) Performed feature engineering (OR) rationale for not doing it [1] 2) Detected and treated the outliers (if any) [1] 3) Splitted the data into train and test sets [1] 4) Defined the preprocessing pipeline for encoding the categorical variables [2] * If outliers are not treated, a clear and logical reasoning should be provided for the same.	1) Splitted the data into train and test sets 2) Defined the preprocessing pipeline for encoding the categorical variables	1) Splitted the data into train and test sets	8.33%
Model Building	- Choose the metric of choice with proper rationale - Build any 2 ML models as a part of a pipeline with the data preprocessing steps - Comment of the performance of the models * The ML models to be built can be any two out of Decision Tree, Bagging, Random Forest, AdaBoost, Gradient Boosting, and XGBoost	7	1) Choose the metric of choice with proper rationale [1] 2) Build any 2 ML models as a part of a pipeline with the data preprocessing steps [2.5 x 2 = 5] 3) Comment of the performance of the models [1]	1) Build any 2 ML models as a part of a pipeline with the data preprocessing steps	1) Build one ML model	11.67%

Model Performance Improvement - Hyperparameter Tuning	- Use hyperparameter tuning to tune the models wrt the metric of choice - Comment of the performance of the tuned models	7	1) Tuned the models using hyperparameter tuning [3 x 2 = 6] 2) Comment of the performance of the tuned models [1]	1) Tuned the models using hyperparameter tuning	1) Tuned one model using hyperparameter tuning	11.67%
Model Performance Comparison, Final Model Selection, and Serialization	- Compare the performances of all models built and choose the best model (with proper rationale) - Check the performance of the best model on the test set - Serialize the best model - Load the serialized model and make predictions on the test set	7	1) Compared model performances of tuned models on both training and validation sets [2] 2) The best model is selected with proper reasoning [1] 3) Test performance checked for the final model [1] 4) Serialized the trained model with the best performance [2] 5) Loaded the serialized model and validated its predictions [1]	1) Compared model performances of tuned models on both training and validation sets 2) The best model is selected 3) Test performance checked for the final model 4) Serialized the trained model with the best performance	1) The best model is selected 2) Serialized the trained model with the best performance	11.67%
Deployment - Backend	- Define the Flask API - Define the dependencies - Define the Dockerfile	6	1) Defined the Flask API [3] 2) Defined the dependencies [1] 3) Defined the Dockerfile [2]	1) Defined the Flask API 2) Defined the dependencies	1) Defined the Flask API	10.00%

Deployment - Frontend and Inference	- Define the Streamlit app - Define the dependencies - Push backend and frontend files to the repository - Define the forwarded URL - Online inference: Single prediction	10	1) Defined the Streamlit app [2] 2) Defined the dependencies [1] 3) Defined the Dockerfile [1] 4) Pushed backend and frontend files to the repository [2] 5) Defined the forwarded URL [2] 6) Online inference: Single prediction [2]	1) Defined the Streamlit app 2) Defined the dependencies 3) Defined the Dockerfile 4) Pushed backend and frontend files to the repository	1) Defined the Streamlit app 2) Defined the dependencies	16.67%
Actionable Insights and Recommendations	- Write down insights from the analysis conducted - Provide actionable business recommendations	4	1) Insights (at least 3) [1x3=3] 2) Recommendations (at least 2) [1x0.5=1]	1) 2 Insights and 1 recommendation shared	1) Some point mentioned but it's irrelevant to the business problem	6.67%
Presentation/Notebook - Overall quality	- Structure and flow - Crispness - Visual appeal - Conclusion and Business Recommendations OR - Structure and flow - Well commented code - Conclusion and Business Recommendations	8	1) Clear structure and flow - everything sits well in a story 2) Crispness (Not too many words, just enough to keep the focus on key things/points) 3) Visual appeal (Use of charts, colors, diagrams, format, symmetry, informative visualizations that are easy to interpret) OR 1) Well structured notebook with a logical flow 2) Clean and well commented code	1) There is structure and flow but some bits are jumbled 2) Points are made but in too many words 3) Lesser charts, format is not the cleanest OR 1) There is structure and flow but some bits are missing 2) Some of the code is commented * If any section is missing and points have been deducted for the same in a previous section, then no points should be deducted in this section	1) No structure or flow 2) Only a few points are covered, story is not complete 3) Not many visuals used OR 1) No structure or flow 2) No comments in the code	13.33%